

Análisis de Patrones de Diseño y Arquetipos

Proyecto Sanos y Salvos — Evaluación Parcial N°2

DSY1106 Desarrollo Fullstack III

Integrante	Rol
Benjamin Arellano Gallardo	Desarrollador Fullstack
Benjamin Valdebenito	Desarrollador Fullstack
Maximiliano Vera	Desarrollador Fullstack
Matias Guzman	Desarrollador Fullstack

1. Introducción

El presente documento describe los patrones de diseño implementados en el proyecto Sanos y Salvos, una plataforma veterinaria desarrollada con arquitectura de microservicios Spring Boot 3.4, frontend React 18 con Vite, MySQL 8.0 como base de datos persistente y Keycloak 24 para autenticación OAuth2. Todo el entorno de infraestructura (MySQL y Keycloak) se levanta mediante Docker Compose.

2. Stack Tecnológico

Componente	Tecnología	Versión
Microservicios	Spring Boot	3.4.5
Service Discovery	Spring Cloud Netflix Eureka	2023.x
API Gateway	Spring Cloud Gateway	2023.x
Base de datos	MySQL (Docker)	8.0
Autenticación	Keycloak (Docker)	24.0.0
Frontend	React + Vite	18.3 / 5.3
HTTP Client	Axios	1.7.2
Enrutamiento	React Router DOM	6.24
Auth Client	Keycloak JS	25.0

3. Patrones de Diseño

3.1 Patrón Builder — msvc-mascotas (:8081)

Descripción: Separa la construcción de un objeto complejo de su representación, permitiendo crear diferentes representaciones con el mismo proceso.

Problema que resuelve:

- El objeto Mascota tiene múltiples atributos opcionales. Sin Builder, los constructores se vuelven ilegibles y propensos a errores de orden de parámetros.

Implementación:

- MascotaBuilder permite construir objetos Mascota mediante method chaining, validando campos requeridos antes de construir el objeto final.
- Base de datos: mascotasdb en MySQL :3306

Justificación:

- Código más legible, facilita pruebas unitarias y permite agregar atributos sin romper código existente.

3.2 Patrón Facade — msvc-reportes (:8082) y BFF (:8090)

Descripción: Provee una interfaz simplificada a un subsistema complejo, reduciendo dependencias entre componentes.

Problema que resuelve:

- El BFF necesita coordinar llamadas a 3 microservicios para componer respuestas. Sin Facade, esta lógica se dispersa en los controladores.

Implementación:

- SanosYSalvosFacade en el BFF orquesta las llamadas a mascotas, reportes y usuarios.
- MascotaServiceFacade en msvc-reportes simplifica el acceso a datos de mascotas. Base de datos: reportesdb en MySQL :3306

Justificación:

- Reduce acoplamiento entre frontend y microservicios. Centraliza la lógica de composición de datos.

3.3 Patrón Adapter — msvc-usuarios (:8083)

Descripción: Permite que interfaces incompatibles trabajen juntas, actuando como puente entre dos interfaces.

Problema que resuelve:

- msvc-usuarios necesita consumir datos de msvc-mascotas pero sus modelos no son directamente compatibles.

Implementación:

- MascotaClientAdapter adapta las respuestas HTTP de mascotas al modelo interno de usuarios mediante un MascotaDTO intermedio. Base de datos: usuariosdb en MySQL :3306

Justificación:

- Desacopla los modelos entre microservicios, permitiendo evolución independiente.

4. Arquetipos Maven

Se crearon dos arquetipos Maven que encapsulan la estructura estándar del proyecto para generar nuevos componentes de forma consistente.

4.1 arquetipo-microservicio

Genera un microservicio Spring Boot con Eureka, JPA+MySQL, capas controller/service/repository y pruebas unitarias JUnit 5 preconfiguradas.

4.2 arquetipo-bff

Genera un BFF con Eureka, WebFlux y estructura client/controller/facade siguiendo el patrón Facade del proyecto.

5. Autenticación con Keycloak

La plataforma integra Keycloak 24 como servidor OAuth2/OpenID Connect. Se levanta mediante Docker con el realm sanosysalvos preconfigurado que se importa automáticamente desde keycloak/realm-export.json.

Propiedad	Valor
Imagen Docker	quay.io/keycloak/keycloak:24.0.0
Puerto	9090
Realm	sanosysalvos
Client ID	sanos-y-salvos-client
Issuer URI	http://localhost:9090/realms/sanosysalvos
Redirect URI	http://localhost:5173/*
Usuario de prueba	usuario-test / admin123
Admin Keycloak	admin / admin (http://localhost:9090/admin)

6. Base de Datos MySQL

Se utiliza MySQL 8.0 como motor de base de datos persistente, levantado mediante Docker. Cada microservicio tiene su propia base de datos aislada, siguiendo el principio de database per service de la arquitectura de microservicios.

Microservicio	Base de datos	Puerto
msvc-mascotas	mascotasdb	3306
msvc-reportes	reportesdb	3306
msvc-usuarios	usuariosdb	3306

Las bases de datos se crean automáticamente al levantar Docker mediante el script mysql/init.sql. Los datos persisten en el volumen mysql_data.

7. Conclusión

Los tres patrones (Builder, Facade, Adapter) resuelven problemas concretos de la arquitectura implementada. La combinación con MySQL persistente, Keycloak OAuth2 y los arquetipos Maven garantiza un sistema desacoplado, mantenible y listo para escalar. Todo el entorno de infraestructura se levanta con un solo comando `docker compose up -d`.